

JARVIS App - One-Page Summary

Evidence basis: README.md, SETUP.md, API.md, jarvis/app.py, jarvis/database.py, jarvis/frontend/vite.config.ts

What it is

JARVIS is a modular enterprise platform that unifies accounting, HR, CRM, marketing, approvals, document workflows, and an AI assistant in one web app.

The repo implements a Flask backend plus a React SPA frontend, with PostgreSQL as the system of record and optional external integrations (for example Google Drive and ANAF e-Factura).

Who it is for

Primary persona explicitly named in repo: Not found in repo.

Inferred from modules/endpoints: finance and operations teams at companies that manage invoices, statements, approvals, employees, and cross-functional business workflows.

What it does

- Handles invoice ingestion/parsing, allocation, lifecycle actions (restore/bin/permanent delete), and accounting summaries.
- Parses and manages bank statements and transactions, with matching/linking to invoices and CSV export.
- Provides role-based access, authentication, permission matrices, organization data, and profile/settings management.
- Runs approval workflows (flows, requests, delegations, multi-step decisions) and notifications.
- Includes CRM and marketing modules for leads/deals/projects, budgets, KPI/OKR style operations.
- Offers AI-enabled capabilities, including AI invoice parsing and an AI agent module with multi-provider support (repo docs).

How it works (architecture)

- Frontend: React 19 + TypeScript + Vite (`jarvis/frontend`) built to `jarvis/static/react`; Flask serves SPA at `/app/\*` and static assets at `/assets/\*`.
- Backend: Flask app (`jarvis/app.py`) registers many blueprints (auth, accounting, HR, CRM, DMS, marketing, approvals, AI, settings, connectors) and exposes JSON APIs.
- Data layer: PostgreSQL accessed via psycopg2 threaded connection pool (`jarvis/database.py`); repository pattern (`core/base\_repository.py`) centralizes query/transaction boilerplate.
- Flow: browser -> React route/API call -> Flask route -> service layer orchestration (for example `InvoiceService`) -> repository SQL -> PostgreSQL -> JSON response to SPA.

How to run (minimal)

1. Install dependencies: `pip install -r requirements.txt`; then `cd jarvis/frontend && npm ci && cd ../..`.
2. Set required env vars: `DATABASE\_URL` and `FLASK\_SECRET\_KEY` (PostgreSQL 14+; enable `pgvector` for RAG features).
3. Start backend: `PORT=5001 DATABASE\_URL="postgresql://user@localhost/defaultdb" FLASK\_SECRET\_KEY="dev-key" python3 jarvis/app.py`.
4. Start frontend dev server: `cd jarvis/frontend && npm run dev`.
5. Open `http://localhost:5173/app/dashboard` (or Flask-served build at `http://localhost:5001/app/dashboard`).